Transformers前夜

自然言語処理特論

正田 備也

masada@rikkyo.ac.jp

言語モデル

言語モデル

- 単語の列$(x_1, x_2, \ldots, x_m)$が得られる確率をモデリングするモデル
- 例：長さ5の文
  - 以下の等式は、条件付き確率の定義を使うだけで、得られる

$$
\begin{aligned}
p(x_1, x_2, x_3, x_4, x_5) &= p(x_5|x_1, x_2, x_3, x_4)p(x_1, x_2, x_3, x_4) \\
&= p(x_5|x_1, x_2, x_3, x_4)p(x_4|x_1, x_2, x_3)p(x_1, x_2, x_3) \\
&= p(x_5|x_1, x_2, x_3, x_4)p(x_4|x_1, x_2, x_3)p(x_3|x_1, x_2)p(x_1, x_2) \\
&= p(x_5|x_1, x_2, x_3, x_4)p(x_4|x_1, x_2, x_3)p(x_3|x_1, x_2)p(x_2|x_1)p(x_1)
\end{aligned}
$$

- どんな単語列の確率でも、p(ある単語|その単語以前に出現する単語の列)という条件付き確率の積で書ける

context

- contextが与えられたとき、その次にどんな単語が現れやすいかを、より正確に予測できる言語モデルが、良い言語モデル

$p(x_m|x_1, x_2, \ldots, x_{m-1})$のモデリング

- n-gramによる近似
  - 以前に出現する単語のうち、直近の$n-1$個だけで条件付け

$$p(x_m|x_1, x_2, \ldots, x_{m-1}) \approx p\left(x_m|x_{m-(n-1)}, x_{m-(n-2)}, \ldots, x_{m-1}\right)$$

  - 例：長さ5の文を3-gramでモデリング

$$\begin{aligned} & p(x_1, x_2, x_3, x_4, x_5) \\ &= p(x_5|x_1, x_2, x_3, x_4)p(x_4|x_1, x_2, x_3)p(x_3|x_1, x_2)p(x_2|x_1)p(x_1) \\ &= p(x_5|x_3, x_4)p(x_4|x_2, x_3)p(x_3|x_1, x_2)p(x_2|x_1)p(x_1) \end{aligned}$$

- できるだけcontextの長さをこのように限定したくない
  - どうすればいい？

深層学習言語モデル以前の最高峰

- The Sequence Memoizer
  - https://dl.acm.org/doi/10.1145/1897816.1897842
  - コンテクスト長が無限の言語モデル

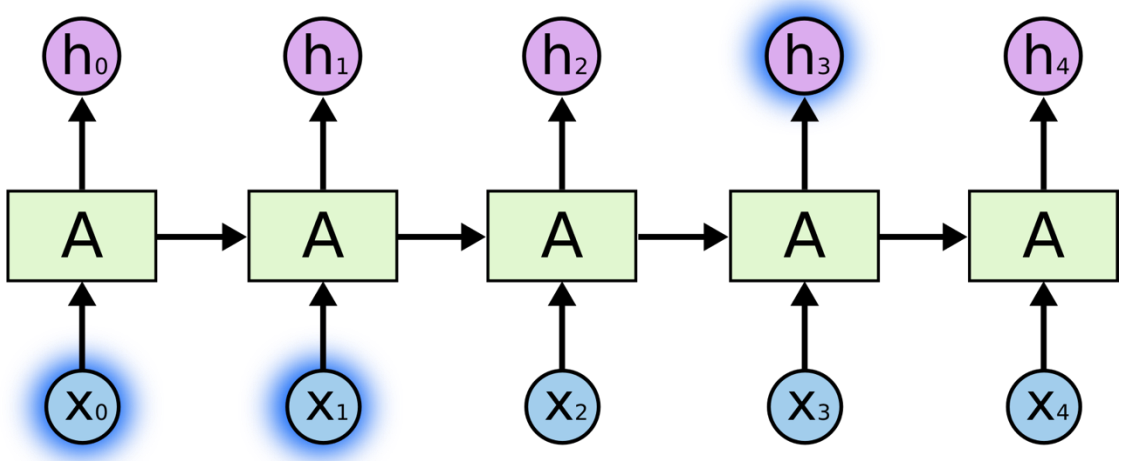
cf. Infini-gram

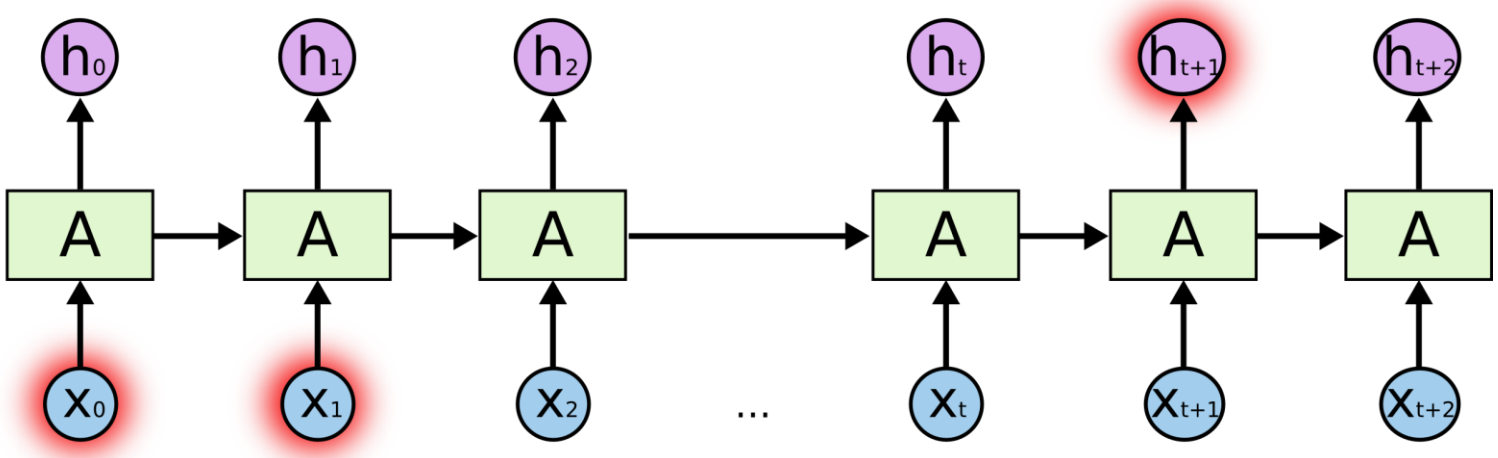
- https://arxiv.org/abs/2401.17377

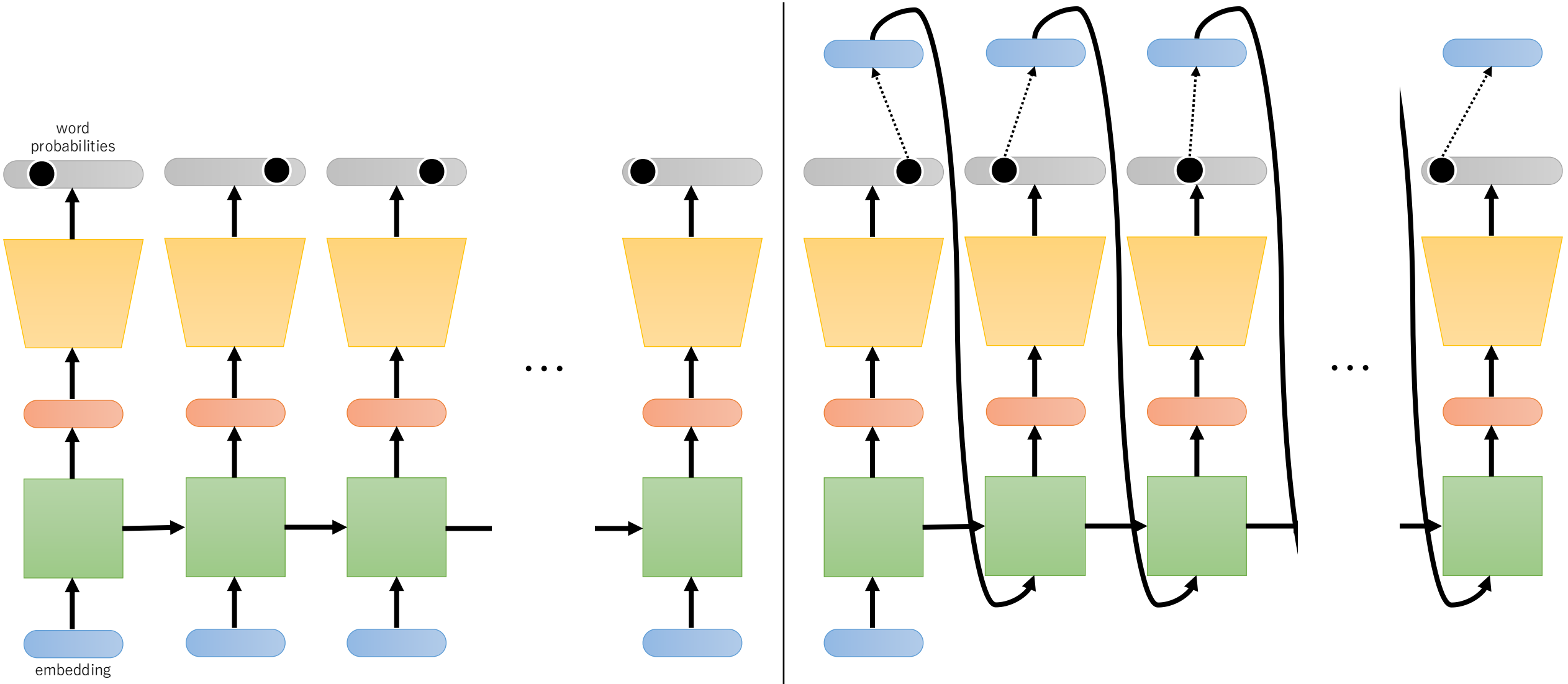
RNN

RNN

https://colah.github.io/posts/2015-08-Understanding-LSTMs/







学習するとき

生成するとき

Learning phrase representations using RNN encoder-decoder for statistical machine translation [Cho+ 14]

The encoder is an RNN that reads each symbol of an input sequence $\mathbf{x}$ sequentially. As it reads each symbol, the hidden state of the RNN changes according to Eq. (1). After reading the end of the sequence (marked by an end-of-sequence symbol), the hidden state of the RNN is a summary $\mathbf{c}$ of the whole input sequence.

The decoder of the proposed model is another RNN which is trained to *generate* the output sequence by predicting the next symbol y_t given the hidden state $\mathbf{h}_{\langle t \rangle}$. However, unlike the RNN described in Sec. 2.1, both y_t and $\mathbf{h}_{\langle t \rangle}$ are also conditioned on y_{t-1} and on the summary $\mathbf{c}$ of the input sequence. Hence, the hidden state of the decoder at time t is computed by,

$$\mathbf{h}_{\langle t \rangle} = f\left(\mathbf{h}_{\langle t-1 \rangle}, y_{t-1}, \mathbf{c}\right),$$

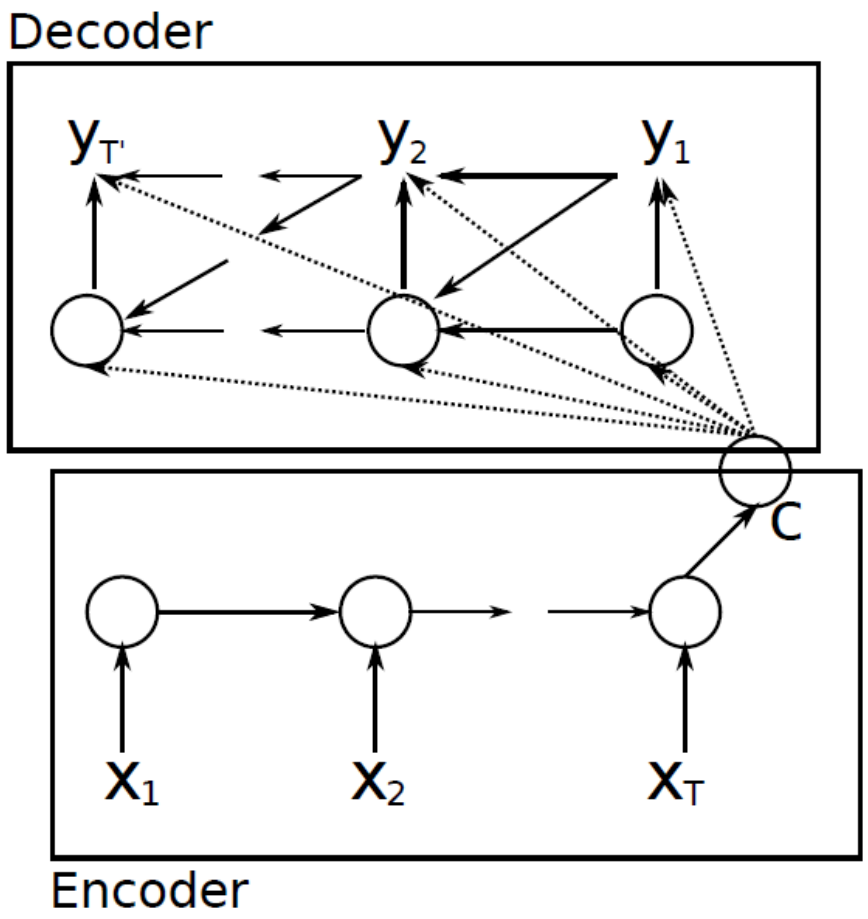
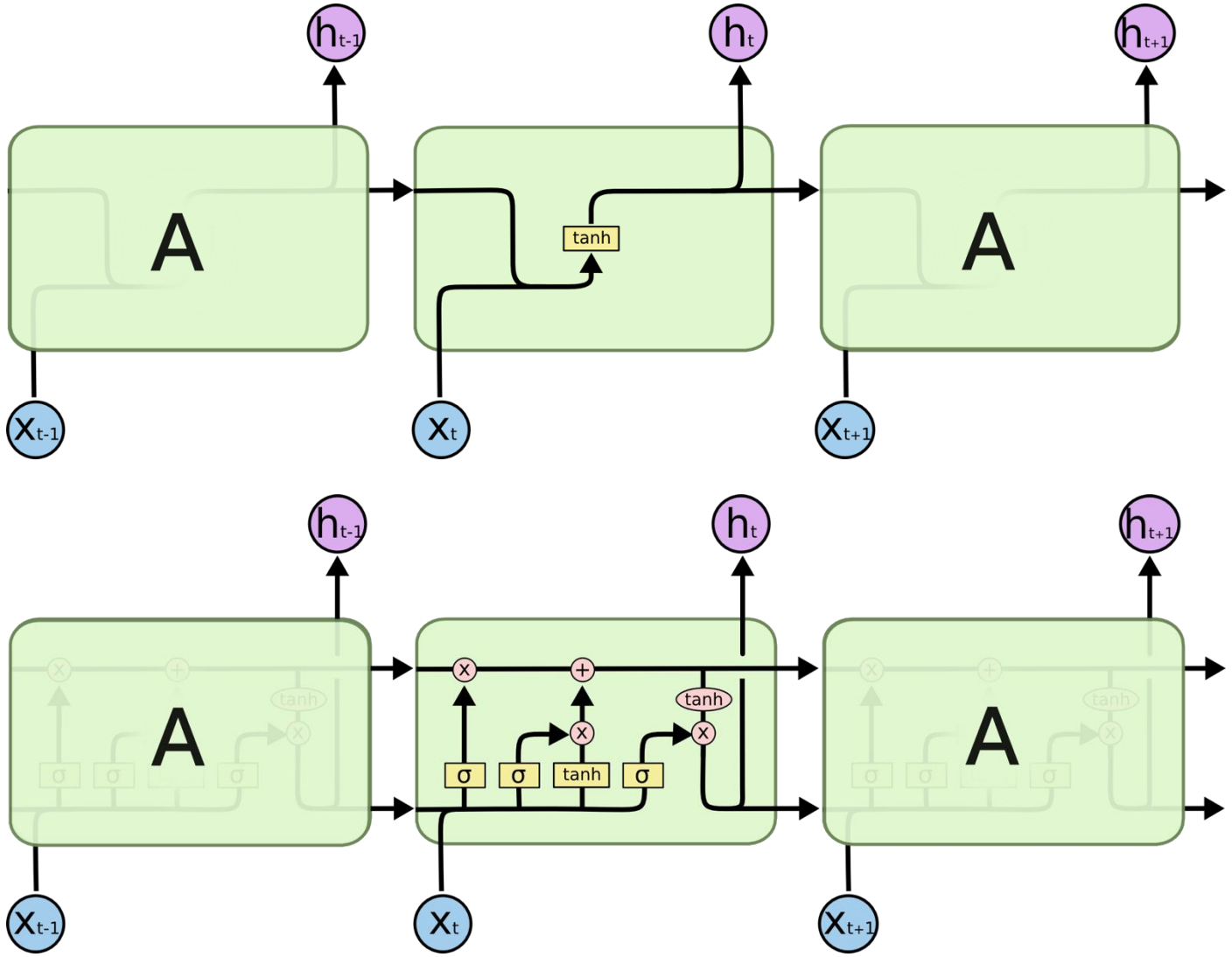


Figure 1: An illustration of the proposed RNN Encoder–Decoder.

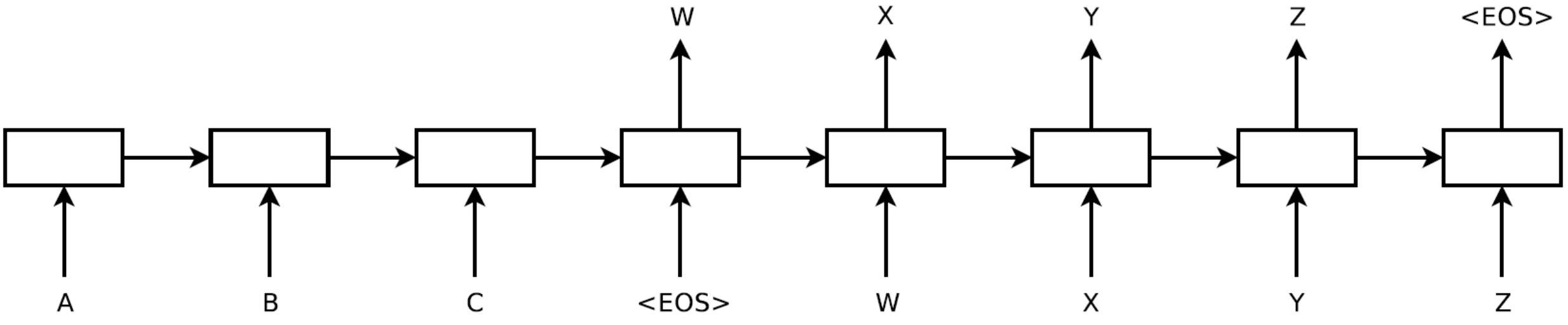
LSTM

https://colah.github.io/posts/2015-08-Understanding-LSTMs/



Sequence to Sequence Learning with Neural Networks [Sutskever+ 14]

Sequences pose a challenge for DNNs because they require that the dimensionality of the inputs and outputs is known and fixed. In this paper, we show that a straightforward application of the Long Short-Term Memory (LSTM) architecture [16] can solve general sequence to sequence problems. The idea is to use one LSTM to read the input sequence, one timestep at a time, to obtain large fixed-dimensional vector representation, and then to use another LSTM to extract the output sequence from that vector (fig. 1). The second LSTM is essentially a recurrent neural network language model [28, 23, 30] except that it is conditioned on the input sequence. The LSTM's ability to successfully learn on data with long range temporal dependencies makes it a natural choice for this application due to the considerable time lag between the inputs and their corresponding outputs (fig. 1).



機械翻訳の定量的評価

- BLEU (Bilingual Evaluation Understudy)
  - BLEU: a Method for Automatic Evaluation of Machine Translation [Papineni+ 02]

- 参考になる動画
  - https://www.youtube.com/watch?v=DejHQYAGb7Q

BLEU: a Method for Automatic Evaluation of Machine Translation [Papineni+ 02]

- Candidateに現れるtheはreferenceにも現れる
- この意味でtheは正しい
- Candidateに含まれる単語のうち、最低1つのreferenceに現れる単語は7つ
- よって、正しい単語の割合は、7/7 = 1
- つまり、このcandidateは100%正しい？？？

- Candidateに含まれる単語が単独のreferenceに現れる回数の最大値を求めておく（theの場合、2回）
- Candidateに含まれる単語のcandidate内での出現頻度を、こうして求めた最大値でクリップする
  - Theは7回現れているが2回とみなす

unigram precision. To compute this, one first counts the maximum number of times a word occurs in any single reference translation. Next, one clips the total count of each candidate word by its maximum reference count,[2] adds these clipped counts up, and divides by the total (unclipped) number of candidate words.

Example 2.

Candidate: the the the the the the the.

Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

Modified Unigram Precision = 2/7.[3]

In Example 1, Candidate 1 achieves a modified unigram precision of 17/18; whereas Candidate 2 achieves a modified unigram precision of 8/14. Similarly, the modified unigram precision in Example 2 is 2/7, even though its standard unigram precision is 7/7.

[2] $Count_{clip} = \min(Count, Max_Ref_Count)$. In other words, one truncates each word's count, if necessary, to not exceed the largest count observed in any single reference for that word.

[3] As a guide to the eye, we have underlined the important words for computing modified precision.

BLEU: a Method for Automatic Evaluation of Machine Translation [Papineni+ 02]

- 複数の文をまとめて評価するときは、micro-averageをとる

A source sentence may translate to many target sentences, in which case we abuse terminology and refer to the corresponding target sentences as a "sentence." We first compute the n-gram matches sentence by sentence. Next, we add the clipped n-gram counts for all the candidate sentences and divide by the number of candidate n-grams in the test corpus to compute a modified precision score, p_n, for the entire test corpus.

$$p_n = \frac{\sum_{C \in \{Candidates\}} \sum_{n\text{-}gram \in C} Count_{clip}(n\text{-}gram)}{\sum_{C' \in \{Candidates\}} \sum_{n\text{-}gram' \in C'} Count(n\text{-}gram')}.$$

BLEU: a Method for Automatic Evaluation of Machine Translation [Papineni+ 02]

- n-gramのmodified precisionは、nが増えるにつれて指数関数的に減少する
- よって、線型結合で平均をとるのは不適切

- そこで、modified precisionの対数の平均をとる

How should we combine the modified precisions for the various n-gram sizes? A weighted linear average of the modified precisions resulted in encouraging results for the 5 systems. However, as can be seen in Figure 2, the modified n-gram precision decays roughly exponentially with n: the modified unigram precision is much larger than the modified bigram precision which in turn is much bigger than the modified trigram precision. A reasonable averaging scheme must take this exponential decay into account; a weighted average of the logarithm of modified precisions satisifies this requirement.

BLEU uses the average logarithm with uniform weights, which is equivalent to using the geometric mean of the modified n-gram precisions.[5,6] Experi-

BLEU: a Method for Automatic Evaluation of Machine Translation [Papineni+ 02]

- 長すぎる翻訳文については、ここまでの議論ですでにペナルティが課されている

- 問題は、短すぎる翻訳文にどのようなペナルティを課するか
  - 次のスライド

One consideration remains: if we computed the brevity penalty sentence by sentence and averaged the penalties, then length deviations on short sentences would be punished harshly. Instead, we compute the brevity penalty over the entire corpus to allow some freedom at the sentence level. We first compute the test corpus' effective reference length, r, by summing the best match lengths for each candidate sentence in the corpus. We choose the brevity penalty to be a decaying exponential in r/c, where c is the total length of the candidate translation corpus.

BLEU: a Method for Automatic Evaluation of Machine Translation [Papineni+ 02]

- 右の式が最終形

We first compute the geometric average of the modified n-gram precisions, p_n, using n-grams up to length N and positive weights w_n summing to one.

Next, let c be the length of the candidate translation and r be the effective reference corpus length. We compute the brevity penalty BP,

$$\mathrm{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}.$$

Then,

$$\mathrm{BLEU} = \mathrm{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right).$$

The ranking behavior is more immediately apparent in the log domain,

$$\log \mathrm{BLEU} = \min(1 - \frac{r}{c}, 0) + \sum_{n=1}^{N} w_n \log p_n.$$

In our baseline, we use $N = 4$ and uniform weights $w_n = 1/N$.

attention

Neural Machine Translation by Jointly Learning to Align and Translate [Bahdanau+ 14]

In a new model architecture, we define each conditional probability in Eq. (2) as:

$$p(y_i|y_1, \ldots, y_{i-1}, \mathbf{x}) = g(y_{i-1}, s_i, c_i), \tag{4}$$

where s_i is an RNN hidden state for time i, computed by

$$s_i = f(s_{i-1}, y_{i-1}, c_i).$$

It should be noted that unlike the existing encoder–decoder approach (see Eq. (2)), here the probability is conditioned on a distinct context vector c_i for each target word y_i.

The context vector c_i depends on a sequence of *annotations* $(h_1, \cdots, h_{T_x})$ to which an encoder maps the input sentence. Each annotation h_i contains information about the whole input sequence with a strong focus on the parts surrounding the i-th word of the input sequence. We explain in detail how the annotations are computed in the next section.

The context vector c_i is, then, computed as a weighted sum of these annotations h_i:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j. \tag{5}$$

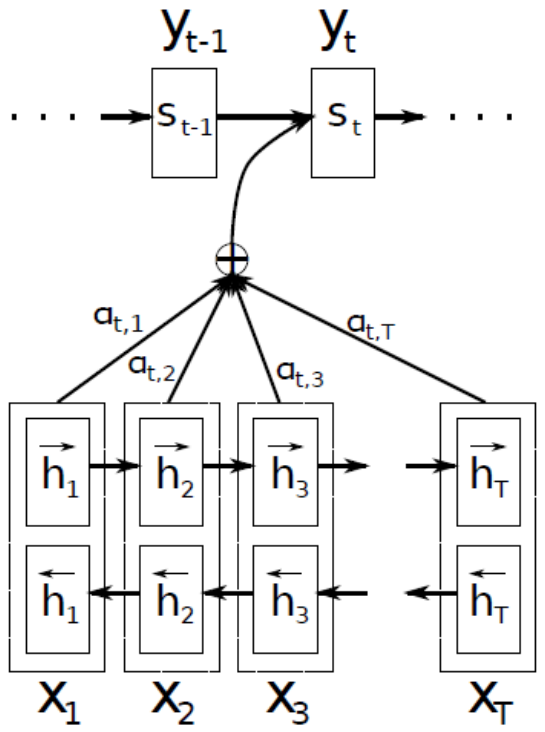


Figure 1: The graphical illustration of the proposed model trying to generate the t-th target word y_t given a source sentence $(x_1, x_2, \ldots, x_T)$.

Neural Machine Translation by Jointly Learning to Align and Translate [Bahdanau+ 14]

The weight α_{ij} of each annotation h_j is computed by

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \tag{6}$$

where

$$e_{ij} = a(s_{i-1}, h_j)$$

is an *alignment model* which scores how well the inputs around position j and the output at position i match. The score is based on the RNN hidden state s_{i-1} (just before emitting y_i, Eq. (4)) and the j-th annotation h_j of the input sentence.

We parametrize the alignment model a as a feedforward neural network which is jointly trained with all the other components of the proposed system. Note that unlike in traditional machine translation,

Effective Approaches to Attention-based Neural Machine Translation [Luong+ 15]

a variable-length alignment vector $\boldsymbol{a}_t$, whose size equals the number of time steps on the source side, is derived by comparing the current target hidden state $\boldsymbol{h}_t$ with each source hidden state $\bar{\boldsymbol{h}}_s$:

$$\begin{aligned} \boldsymbol{a}_t(s) &= \text{align}(\boldsymbol{h}_t, \bar{\boldsymbol{h}}_s) \qquad (7) \\ &= \frac{\exp\left(\text{score}(\boldsymbol{h}_t, \bar{\boldsymbol{h}}_s)\right)}{\sum_{s'} \exp\left(\text{score}(\boldsymbol{h}_t, \bar{\boldsymbol{h}}_{s'})\right)} \end{aligned}$$

Here, score is referred as a *content-based* function for which we consider three different alternatives:

$$\text{score}(\boldsymbol{h}_t, \bar{\boldsymbol{h}}_s) = \begin{cases} \boldsymbol{h}_t^\top \bar{\boldsymbol{h}}_s & \textit{dot} \\ \boldsymbol{h}_t^\top \boldsymbol{W}_{\boldsymbol{a}} \bar{\boldsymbol{h}}_s & \textit{general} \\ \boldsymbol{v}_a^\top \tanh\left(\boldsymbol{W}_{\boldsymbol{a}}[\boldsymbol{h}_t; \bar{\boldsymbol{h}}_s]\right) & \textit{concat} \end{cases}$$

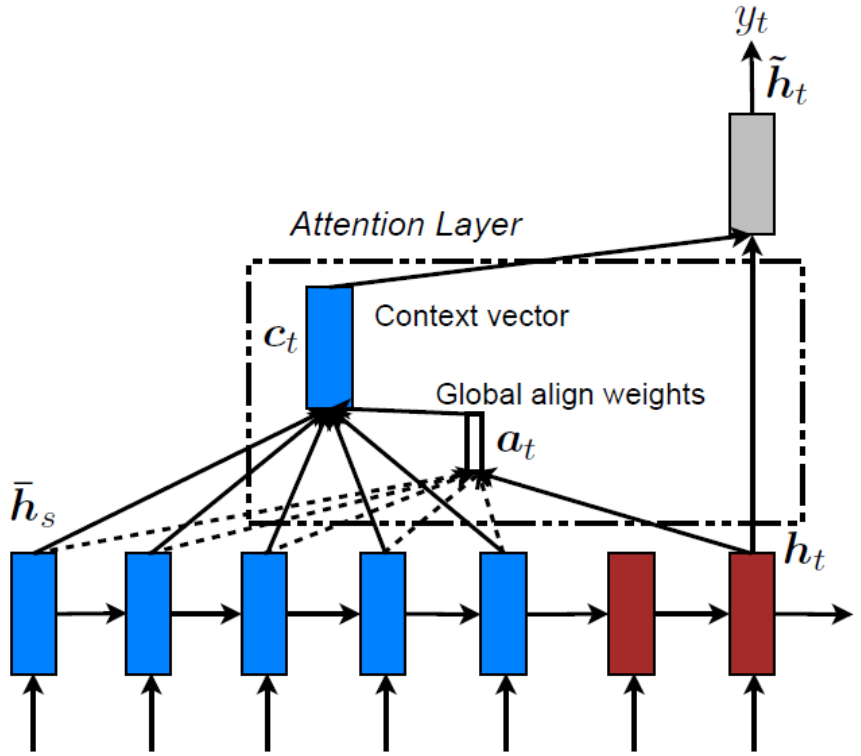


Figure 2: **Global attentional model** – at each time step t, the model infers a *variable-length* alignment weight vector $\boldsymbol{a}_t$ based on the current target state $\boldsymbol{h}_t$ and all source states $\bar{\boldsymbol{h}}_s$. A global context vector $\boldsymbol{c}_t$ is then computed as the weighted average, according to $\boldsymbol{a}_t$, over all the source states.

Effective Approaches to Attention-based Neural Machine Translation [Luong+ 15]

System	Ppl	BLEU	
		Before	After unk
global (location)	6.4	18.1	19.3 (+1.2)
global (dot)	6.1	18.6	20.5 (+1.9)
global (general)	6.1	17.3	19.1 (+1.8)
local-m (dot)	>7.0	x	x
local-m (general)	6.2	18.6	20.4 (+1.8)
local-p (dot)	6.6	18.0	19.6 (+1.9)
local-p (general)	**5.9**	**19**	**20.9 (+1.9)**

Table 4: **Attentional Architectures** – performances of different attentional models. We trained two local-m (dot) models; both have ppl > 7.0.

not learn good alignments. The *global (location)* model can only obtain a small gain when performing unknown word replacement compared to using other alignment functions.[14] For *content-based* functions, our implementation *concat* does not yield good performances and more analysis should be done to understand the reason.[15] It is interesting to observe that *dot* works well for the global attention and *general* is better for the local attention. Among the different models, the local attention model with predictive alignments (*local-p*) is best, both in terms of perplexities and BLEU.

Massive Exploration of Neural Machine Translation Architectures [Britz+ 17]

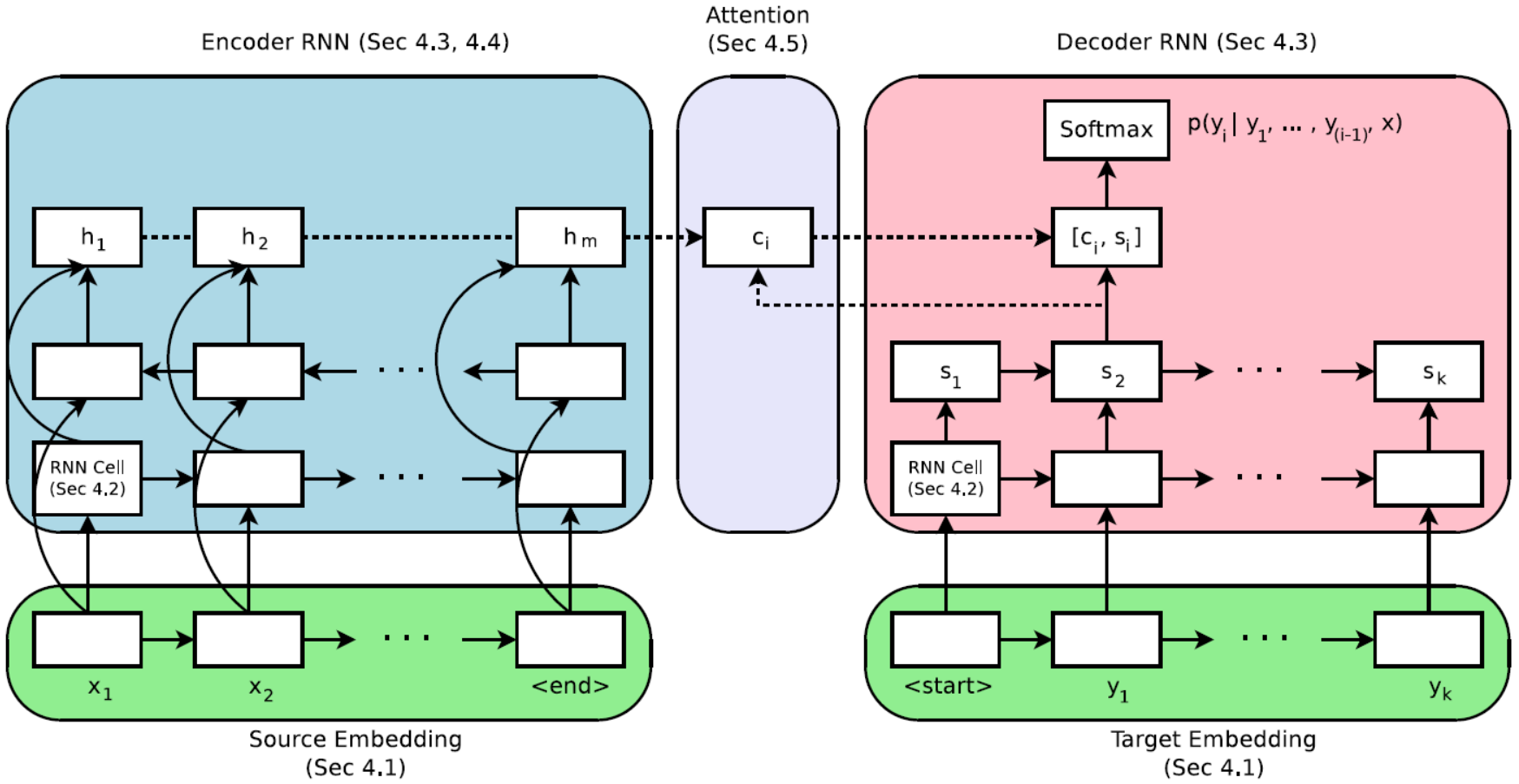


Figure 1: Encoder-Decoder architecture with attention module. Section numbers reference experiments corresponding to the components.

unsupervised pre-training

Semi-supervised Sequence Learning [Dai+ 15]

In our experiments on document classification with 20 Newsgroups [16] and DBpedia [19], and sentiment analysis with IMDB [21] and Rotten Tomatoes [25], LSTMs pretrained by recurrent language models or sequence autoencoders are usually better than LSTMs initialized randomly. This pretraining helps LSTMs reach or surpass previous baselines on these datasets without additional data.

Natural Language Processing (almost) from Scratch [Collobert+ 11]

実質的にfine-tuningの提案になっている

the elusive goals of Artificial Intelligence.

In this contribution, we try to excel on *multiple benchmarks* while *avoiding task-specific engineering*. Instead we use a *single learning system* able to discover adequate internal representations. In fact we view the benchmarks as indirect measurements of the relevance of the internal representations discovered by the learning procedure, and we posit that these intermediate representations are more general than any of the benchmarks. Our desire to avoid task-specific engineered features prevented us from using a large body of linguistic knowledge. Instead we reach good performance levels in most of the tasks by transferring intermediate representations discovered on large unlabeled data sets. We call this approach "almost from scratch" to emphasize the reduced (but still important) reliance on a priori NLP knowledge.

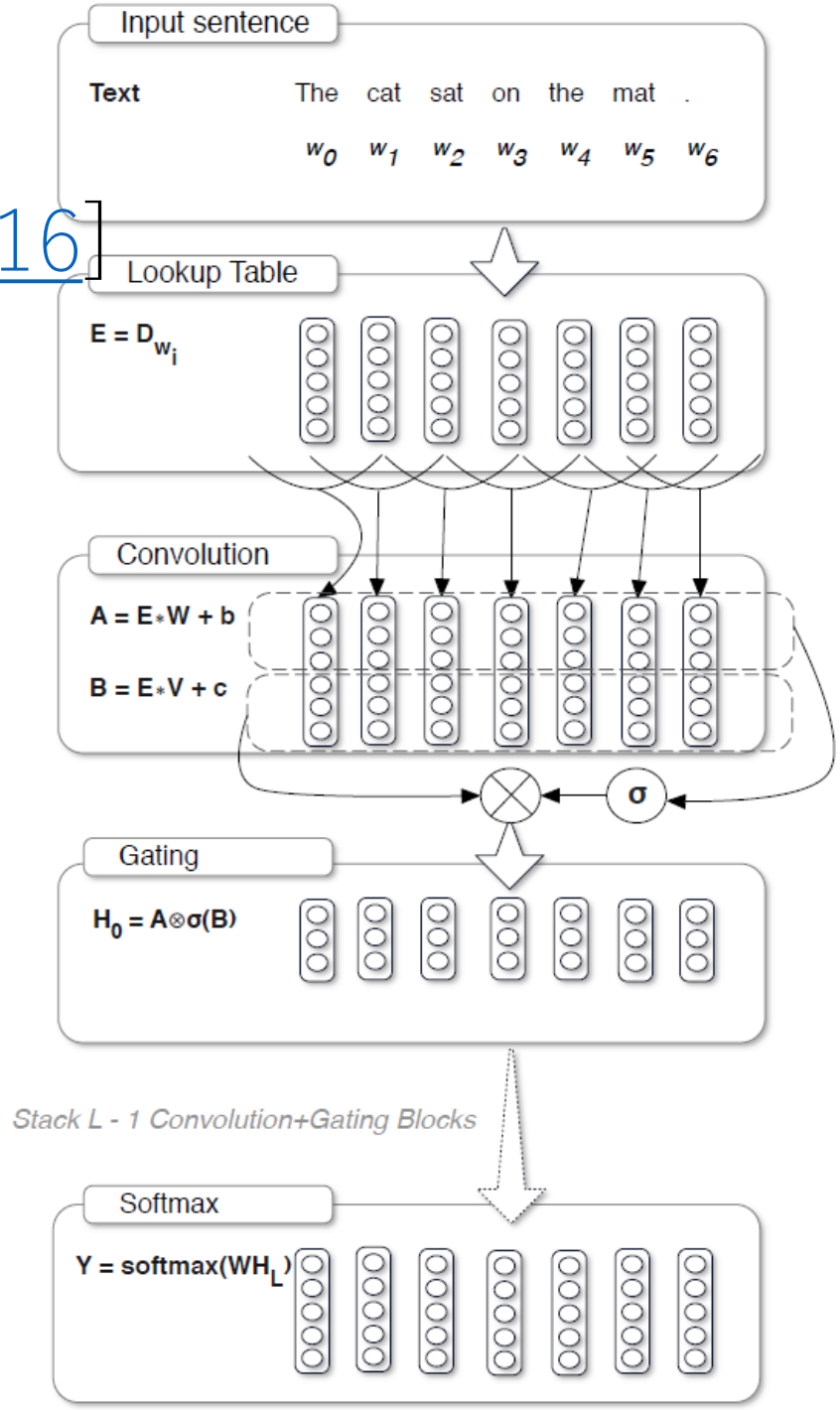
The paper is organized as follows. Section 2 describes the benchmark tasks of interest. Sec-

RNN → CNN

Language Modeling with Gated Convolutional Networks [Dauphin+ 16]

Modern hardware is well suited to models that are highly parallelizable. In recurrent networks, the next output depends on the previous hidden state which does not enable parallelization over the elements of a sequence. Convolutional networks, however, are very amenable to this computing paradigm since the computation of all input words can be performed simultaneously (§2).

Our proposed approach convolves the inputs with a function f to obtain $\mathbf{H} = f * w$ and therefore has no temporal dependencies, so it is easier to parallelize over the individual words of a sentence. This process will compute each context as a function of a number of preceding words. Compared to recurrent networks, the context size is finite but we will demonstrate both that infinite contexts are not necessary and our models can represent large enough contexts to perform well in practice (§5).

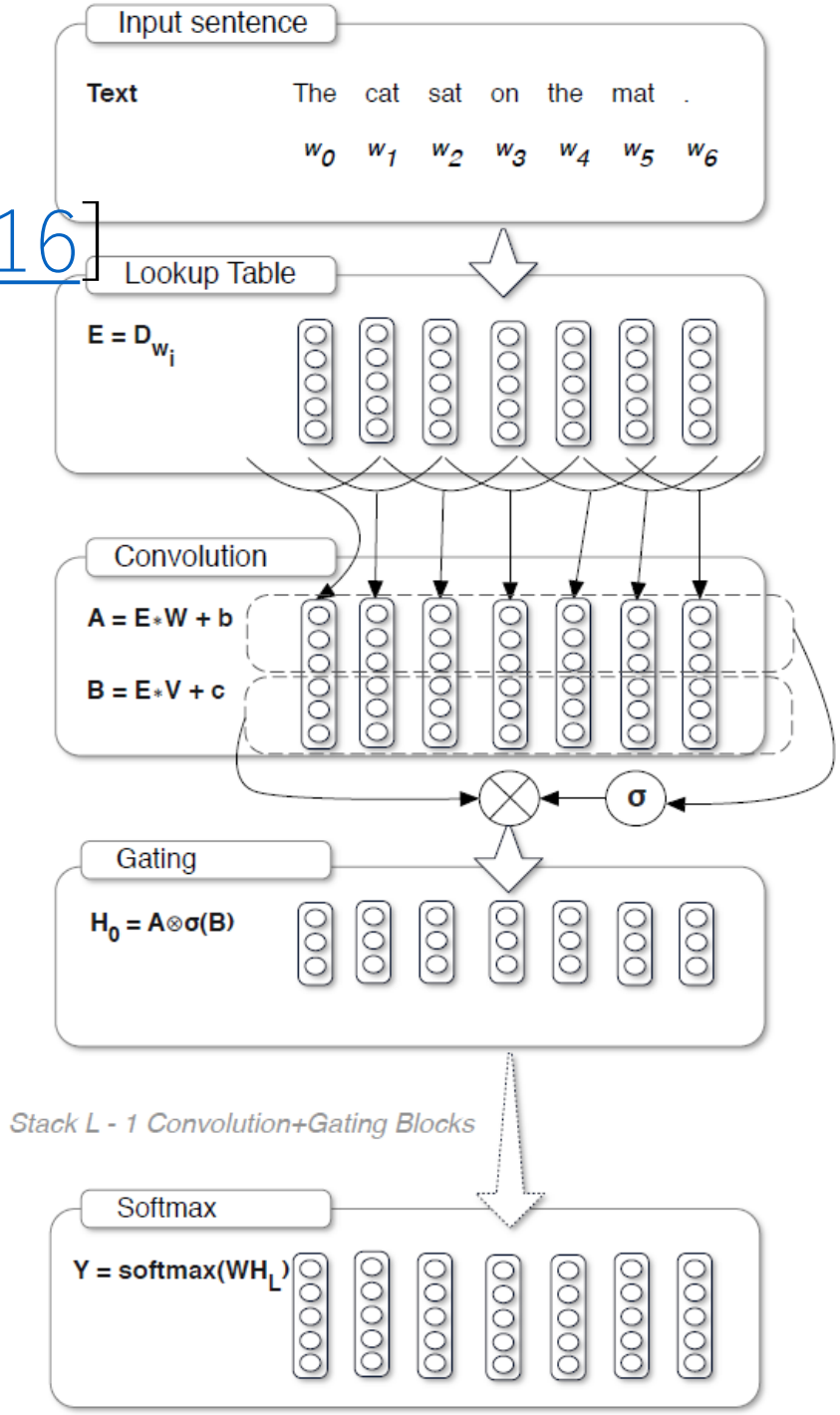


Language Modeling with Gated Convolutional Networks [Dauphin+ 16]

Figure 1 illustrates the model architecture. Words are represented by a vector embedding stored in a lookup table $\mathbf{D}^{|\mathcal{V}| \times e}$ where $|\mathcal{V}|$ is the number of words in the vocabulary and e is the embedding size. The input to our model is a sequence of words $w_0, \ldots, w_N$ which are represented by word embeddings $\mathbf{E} = [\mathbf{D}_{w_0}, \ldots, \mathbf{D}_{w_N}]$. We compute the hidden layers $h_0, \ldots, h_L$ as

$$h_l(\mathbf{X}) = (\mathbf{X} * \mathbf{W} + \mathbf{b}) \otimes \sigma(\mathbf{X} * \mathbf{V} + \mathbf{c}) \qquad (1)$$

where m, n are respectively the number of input and output feature maps and k is the patch size, $\mathbf{X} \in \mathbb{R}^{N \times m}$ is the input of layer h_l (either word embeddings or the outputs of previous layers), $\mathbf{W} \in \mathbb{R}^{k \times m \times n}$, $\mathbf{b} \in \mathbb{R}^n$, $\mathbf{V} \in \mathbb{R}^{k \times m \times n}$, $\mathbf{c} \in \mathbb{R}^n$ are learned parameters, σ is the sigmoid function and $\otimes$ is the element-wise product between matrices.



Convolutional Sequence to Sequence Learning [Gehring+ 17]

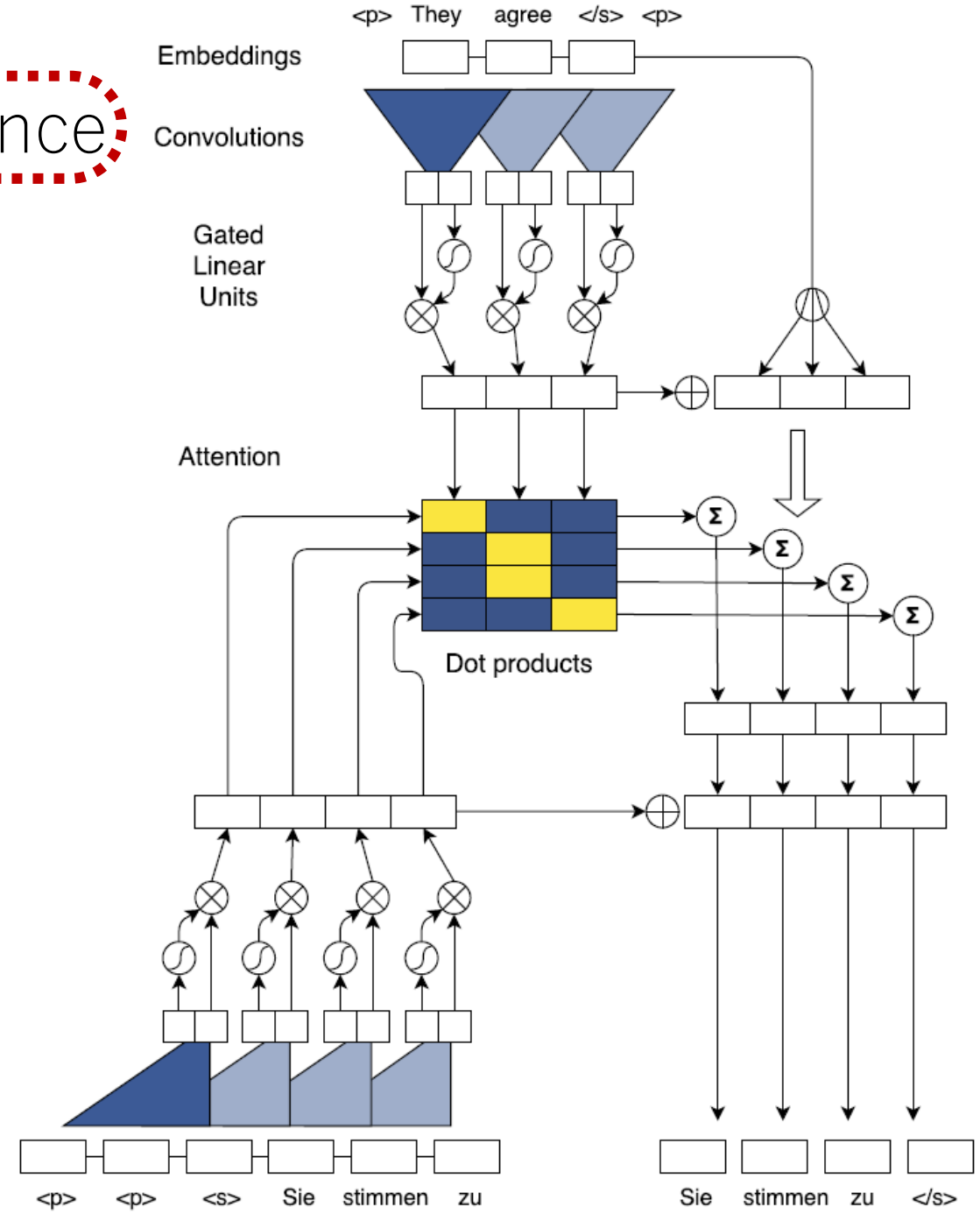
$$d_i^l = W_d^l h_i^l + b_d^l + g_i \quad (1)$$

For decoder layer l the attention a_{ij}^l of state i and source element j is computed as a dot-product between the decoder state summary d_i^l and each output z_j^u of the last encoder block u:

$$a_{ij}^l = \frac{\exp\left(d_i^l \cdot z_j^u\right)}{\sum_{t=1}^{m} \exp\left(d_i^l \cdot z_t^u\right)}$$

The conditional input c_i^l to the current decoder layer is a weighted sum of the encoder outputs as well as the input element embeddings e_j (Figure 1, center right):

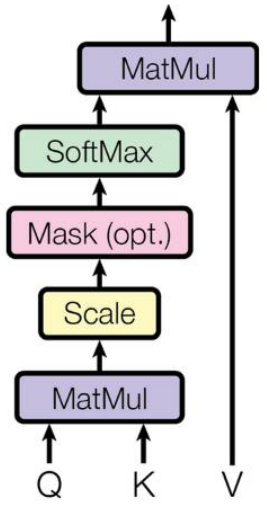
$$c_i^l = \sum_{j=1}^{m} a_{ij}^l (z_j^u + e_j) \quad (2)$$



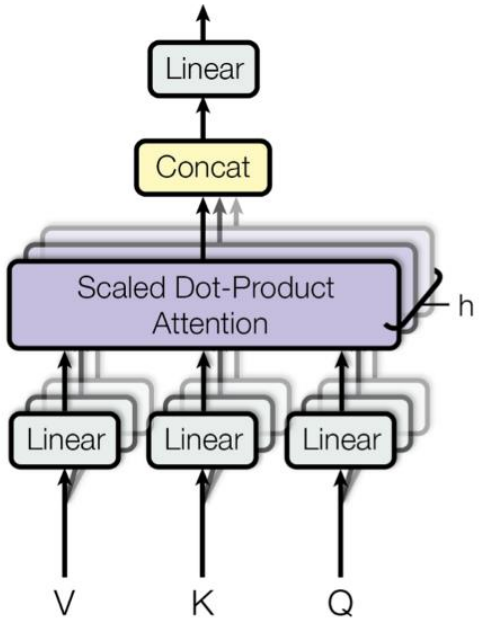
RNN, CNN → Attention

Attention Is All You Need [Vaswani+ 17]

Scaled Dot-Product Attention



Multi-Head Attention



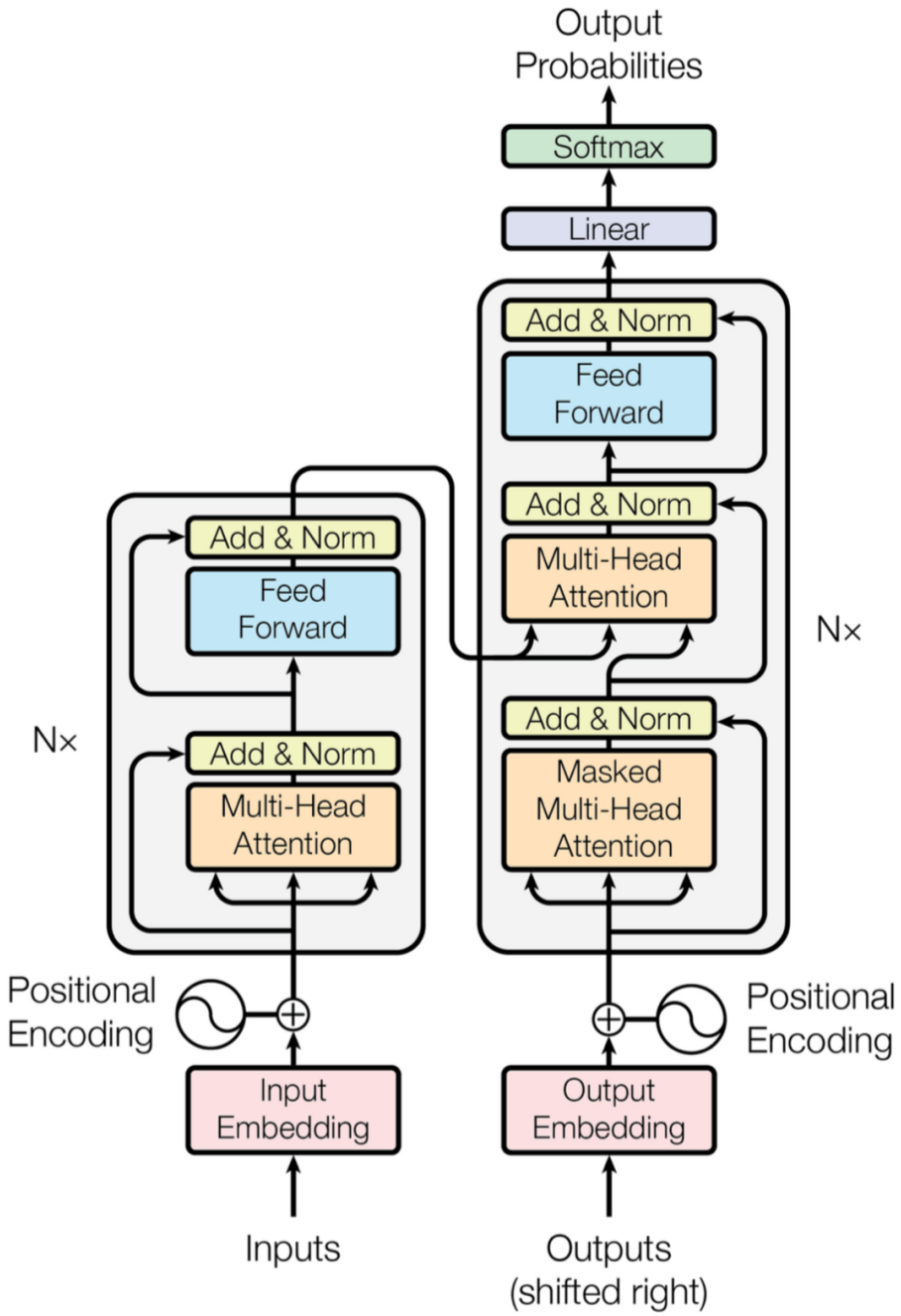


Figure 1: The Transformer - model architecture.

http://jalammar.github.io/illustrated-transformer/

Attention Is All You Need [Vaswani+ 17]

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \quad (1)$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

Attention Is All You Need [Vaswani+ 17]

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [38, 2, 9].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

Attention Is All You Need [Vaswani+ 17]

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d, which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [38] and byte-pair [31] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in

encoder+decoder → decoder

Generating Wikipedia by Summarizing Long Sequences [Liu+ 18]

後で出てくるGPTで使われている
アーキテクチャがこの論文で
提案されている

4.2.3 TRANSFORMER DECODER (T-D)

We introduce a simple but effective modification to T-ED for long sequences that drops the encoder module (almost reducing model parameters by half for a given hyper-parameter set), combines the input and output sequences into a single "sentence" and is trained as a standard language model.

That is, we convert a sequence-transduction example $(m^1, ..., m^n) \mapsto (y^1, ..., y^\eta)$ into the sentence $(w^1, ..., w^{n+\eta+1}) = (m^1, ..., m^n, \delta, y^1, ..., y^\eta)$, where δ is a special separator token and train a model to predict the next word given the previous ones:

$$p(w^1, ..., w^{n+\eta}) = \prod_{j=1}^{n+\eta} p(w^i | w^1, ..., w^{j-1})$$

Since the model is forced to predict the next token in the input, m, as well as y, error signals are propagated from both input and output time-steps during training. We also suspect that for monolingual text-to-text tasks redundant information is re-learned about language in the encoder and decoder. We believe this allows for easier optimization and empirically observe this with longer sequences (see Section 5.3). Note that because of the self-attention of the Transformer, when generating the next token, attention from both m and y are considered. At inference we provide the input sequence, m_i, initially, and auto-regressively generate the output, y_i, as normal.

Improving Language Understanding by Generative Pre-Training [Radford+ 18]

GPT

3.1 Unsupervised pre-training

Given an unsupervised corpus of tokens $\mathcal{U} = \{u_1, \ldots, u_n\}$, we use a standard language modeling objective to maximize the following likelihood:

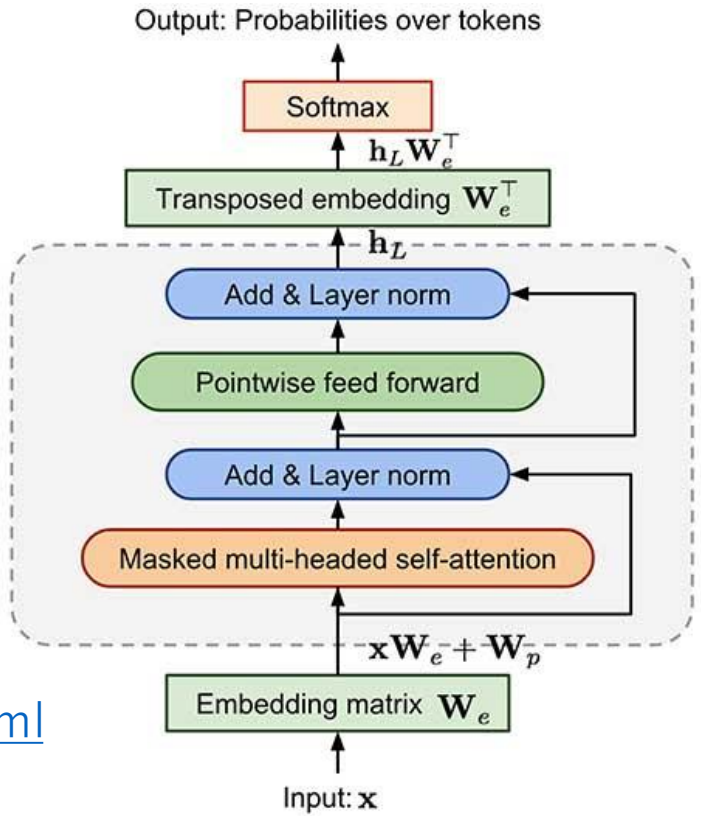
$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \ldots, u_{i-1}; \Theta) \quad (1)$$

where k is the size of the context window, and the conditional probability P is modeled using a neural network with parameters Θ. These parameters are trained using stochastic gradient descent [51].

In our experiments, we use a multi-layer *Transformer decoder* [34] for the language model, which is a variant of the transformer [62]. This model applies a multi-headed self-attention operation over the input context tokens followed by position-wise feedforward layers to produce an output distribution over target tokens:

$$\begin{aligned} h_0 &= UW_e + W_p \\ h_l &= \texttt{transformer_block}(h_{l-1}) \forall i \in [1, n] \\ P(u) &= \texttt{softmax}(h_n W_e^T) \end{aligned} \quad (2)$$

where $U = (u_{-k}, \ldots, u_{-1})$ is the context vector of tokens, n is the number of layers, W_e is the token embedding matrix, and W_p is the position embedding matrix.



https://lilianweng.github.io/lil-log/2019/01/31/generalized-language-models.html

Improving Language Understanding by Generative Pre-Training [Radford+ 18]

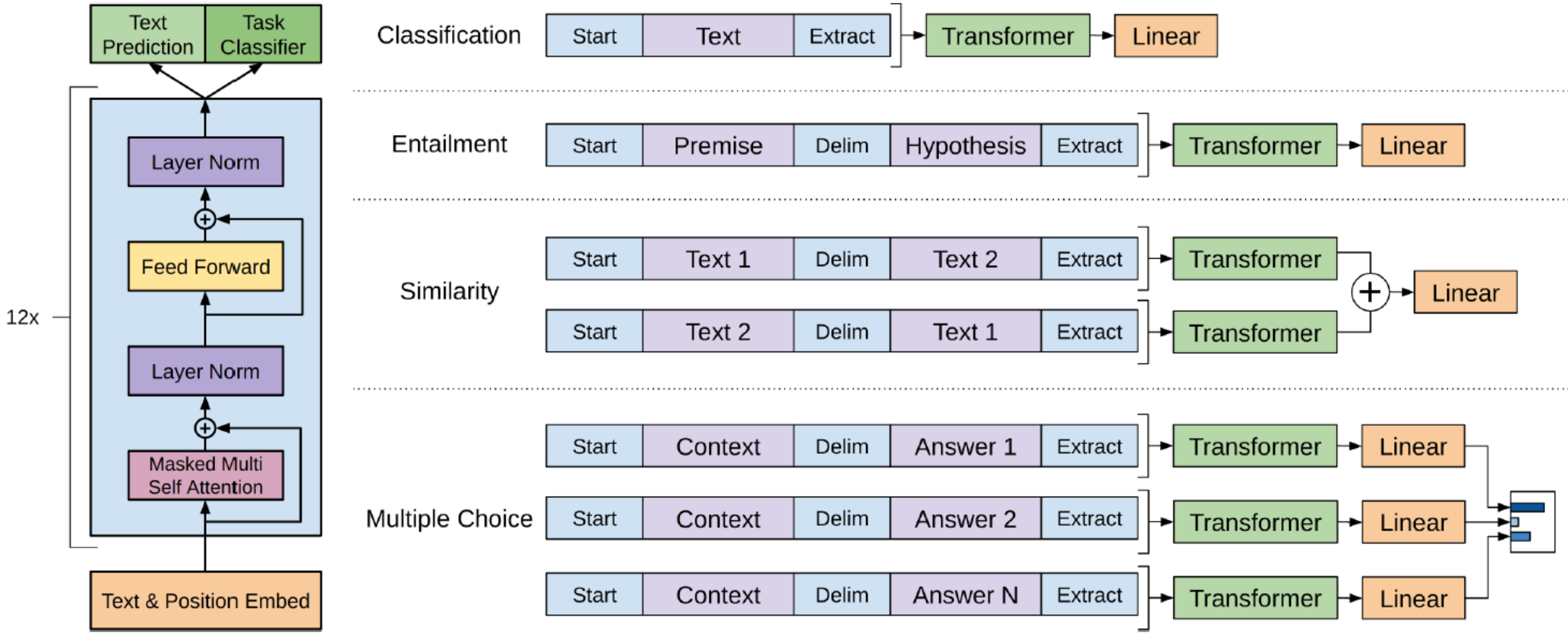


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

autoregressive → autoencoding



Summary of the models

This is a summary of the models available in the transformers library. It assumes you're familiar with the original transformer model. For a gentle introduction check the annotated transformer. Here we focus on the high-level differences between the models. You can check them more in detail in their respective documentation. Also checkout the pretrained model page to see the checkpoints available for each type of model and all the community models.

Each one of the models in the library falls into one of the following categories:

- Autoregressive models
- Autoencoding models
- Sequence-to-sequence models
- Multimodal models

Autoregressive models are pretrained on the classic language modeling task: guess the next token having read all the previous ones. They correspond to the decoder of the original transformer model, and a mask is used on top of the full sentence so that the attention heads can only see what was before in the next, and not what's after. Although those models can be fine-tuned and achieve great results on many tasks, the most natural application is text generation. A typical example of such models is GPT.

Autoencoding models are pretrained by corrupting the input tokens in some way and trying to reconstruct the original sentence. They correspond to the encoder of the original transformer model in the sense that they get access to the full inputs without any mask. Those models usually build a bidirectional representation of the whole sentence. They can be fine-tuned and achieve great results on many tasks such as text generation, but their most natural application is sentence classification or token classification. A typical example of such models is BERT.

Note that the only difference between autoregressive models and autoencoding models is in the way the model is pretrained. Therefore, the same architecture can be used for both autoregressive and autoencoding models. When a given model has been used for both pretraining, we have put it in the category corresponding to the article it was first introduced.

Summary of the models

This is a summary of the models available in 🤗 Transformers. It assumes you're familiar with the original transformer model. For a gentle introduction check the annotated transformer. Here we focus on the high-level differences between the models. You can check them more in detail in their respective documentation. Also check out the pretrained model page to see the checkpoints available for each type of model and all the community models.

Each one of the models in the library falls into one of the following categories:

- autoregressive-models
- autoencoding-models
- seq-to-seq-models
- multimodal-models
- retrieval-based-models



BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Devlin+ 18]

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers of the Transformer (Vaswani et al., 2017). Such restrictions are sub-optimal for sentence-level tasks, and could be very harmful when applying fine-tuning based approaches to token-level tasks such as question answering, where it is crucial to incorporate context from both directions.

In this paper, we improve the fine-tuning based approaches by proposing BERT: **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. BERT alleviates the previously mentioned unidirectionality constraint by using a "masked language model" (MLM) pre-training objective, inspired by the Cloze task (Taylor, 1953). The masked language model randomly masks some of the tokens from the input, and the objective is to predict the original vocabulary id of the masked

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Devlin+ 18]

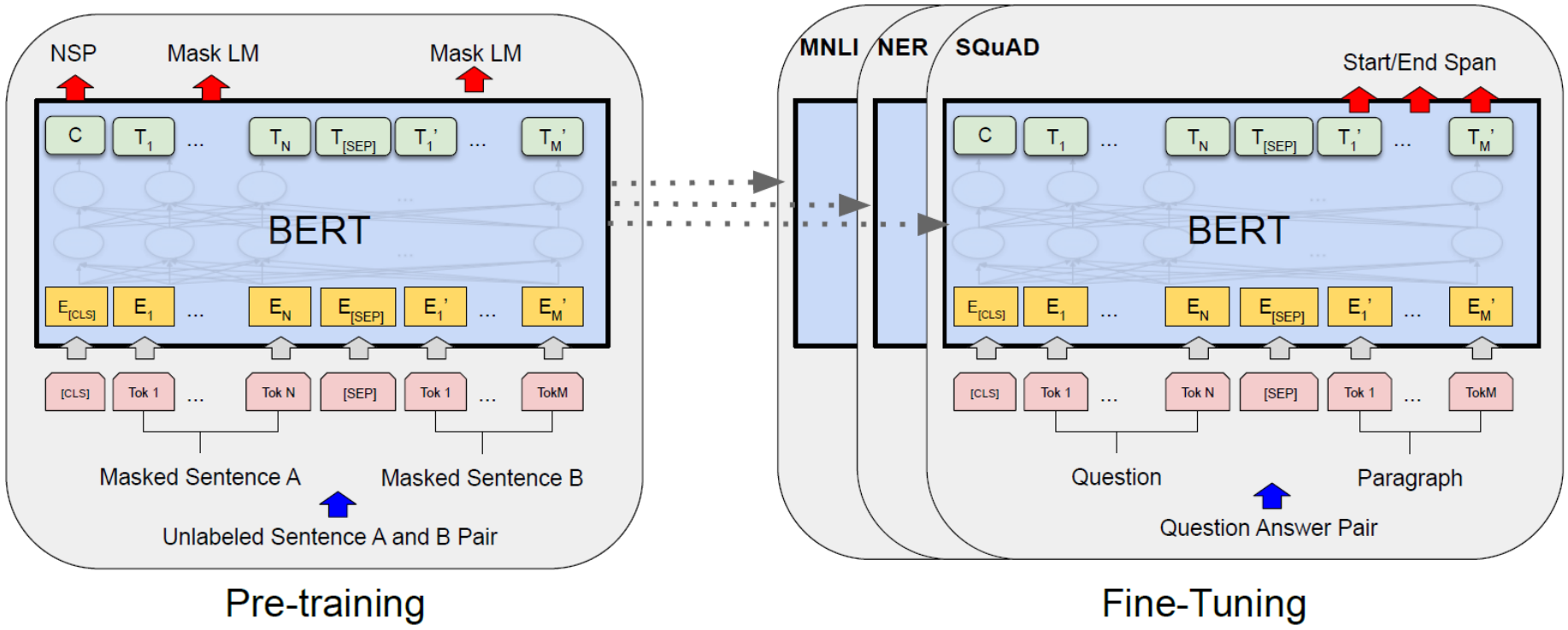


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. `[CLS]` is a special symbol added in front of every input example, and `[SEP]` is a special separator token (e.g. separating questions/answers).

fine-tuning → zero-shot

Language Models are Unsupervised Multitask Learners [Radford+ 19]

GPT-2

In this paper, we connect these two lines of work and continue the trend of more general methods of transfer. We demonstrate language models can perform down-stream tasks in a zero-shot setting – without any parameter or architecture modification. We demonstrate this approach shows potential by highlighting the ability of language models to perform a wide range of tasks in a zero-shot setting. We achieve promising, competitive, and state of the art results depending on the task.

Language Models are Few-Shot Learners [Brown+ 20]

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  cheese =>                           ← prompt
```

Language Models are Few-Shot Learners [Brown+ 20]

One-shot

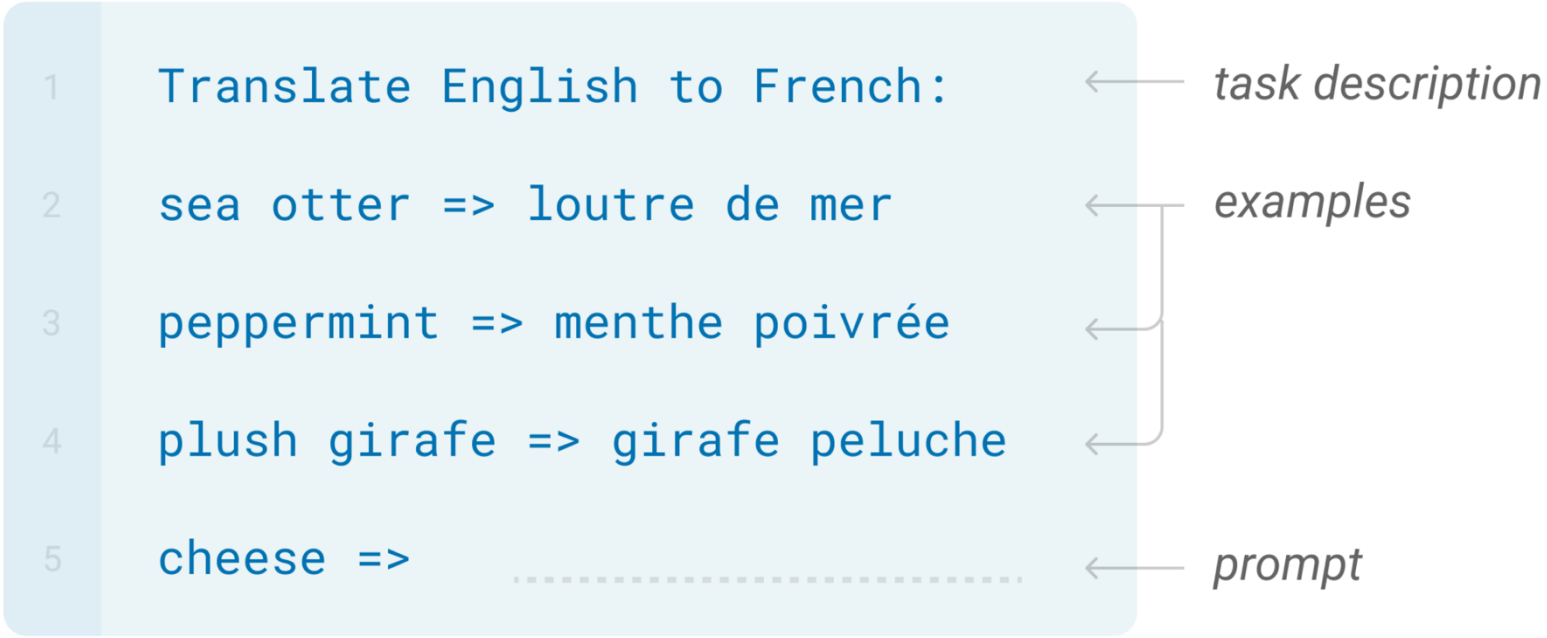
In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← example
3  cheese =>                           ← prompt
```

Language Models are Few-Shot Learners [Brown+ 20]

Few-shot

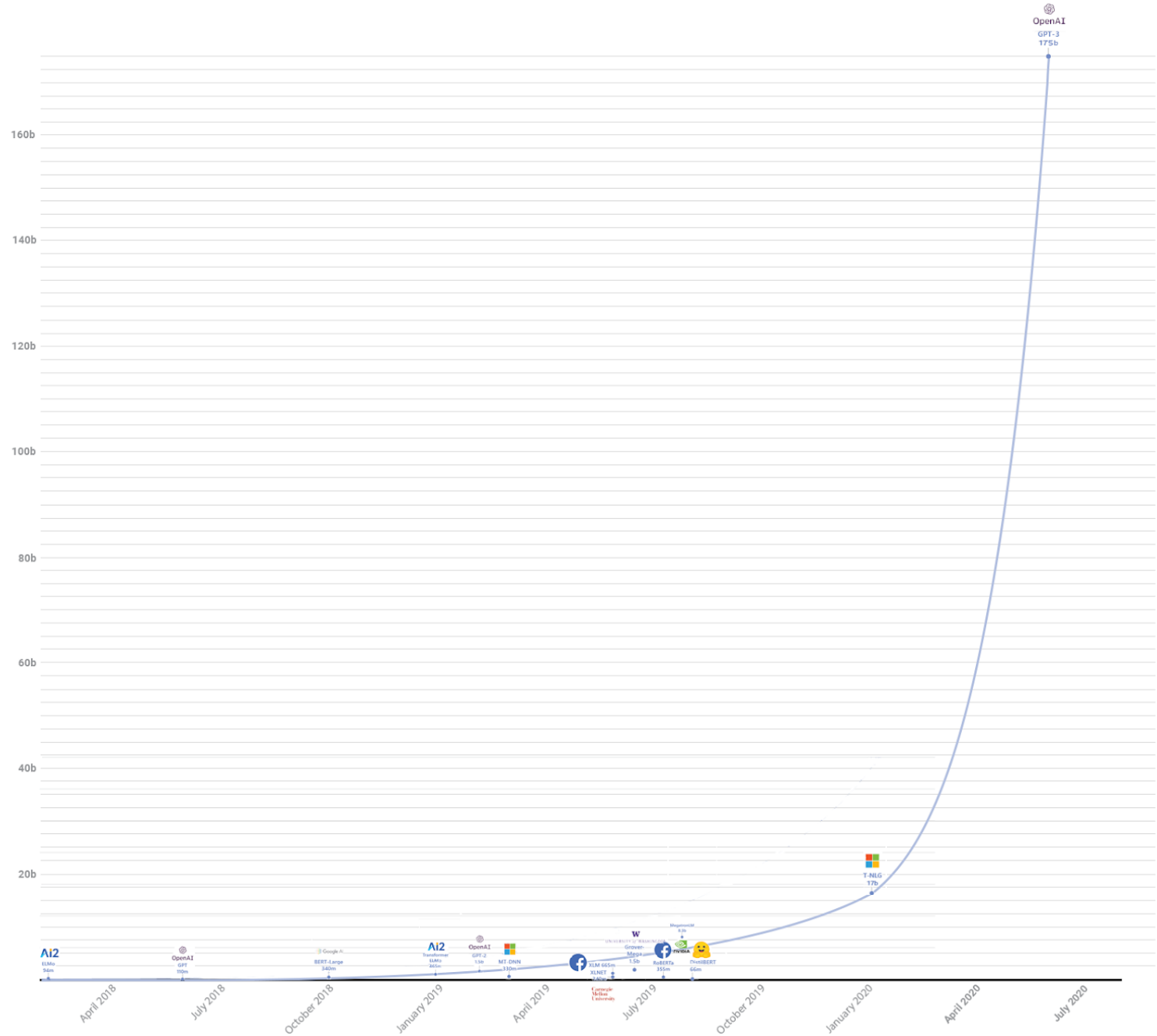
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



Language Models are Unsupervised Multitask Learners [Radford+ 19]

”I’m not the cleverest man in the world, but like they say in French: **Je ne suis pas un imbecile [I’m not a fool].**
In a now-deleted post from Aug. 16, Soheil Eid, Tory candidate in the riding of Joliette, wrote in French: ”**Mentez mentez, il en restera toujours quelque chose**,” which translates as, ”**Lie lie and something will always remain.**”
“I hate the word ‘**perfume**,”’ Burr says. ‘It’s somewhat better in French: ‘**parfum**.’
If listened carefully at 29:55, a conversation can be heard between two guys in French: “**-Comment on fait pour aller de l’autre coté? -Quel autre coté?**”, which means “**- How do you get to the other side? - What side?**”.
If this sounds like a bit of a stretch, consider this question in French: **As-tu aller au cinéma?**, or **Did you go to the movies?**, which literally translates as Have-you to go to movies/theater?
“**Brevet Sans Garantie Du Gouvernement**”, translated to English: “**Patented without government warranty**”.

Table 1. Examples of naturally occurring demonstrations of English to French and French to English translation found throughout the WebText training set.



https://www.productsolving.com/p/openais-gpt-3-will-change-how-we

Priming

https://www.productsolving.com/p/openais-gpt-3-will-change-how-we

How does it work?

One important characteristic of GPT-3 is that it requires priming to work. Priming is when you expose someone (or something) to a specific stimulus that then results in the identification of other things that were previously unnoticed. Here's an example:

> *Imagine you are in the market for a new car. You want to be original but not too flashy, so you start looking for a Volvo. You poke around on Carvana, Volvo.com, and a few other car sites before deciding on a model you like. You still don't feel quite ready to buy, so you put it on the back burner and hop in the car to get lunch. On your way to the fast-food joint down the street, you notice the exact car you were just looking at online! "What a coincidence!" you think to yourself. "I never see any Volvos out on the road, let alone the exact model I was just looking at." Throughout the rest of your trip, you notice four more Volvos. This makes you wonder if you were original at all by assuming a Volvo was a less-than-popular car.*

Priming

https://www.productsolving.com/p/openais-gpt-3-will-change-how-we

I am sure this has happened to you, if not a car then with something else. Nobody read your search history and bought a bunch of Volvos just to mess you with you. You just weren't looking for them before. This is priming. Your initial stimulus was the decision that a Volvo is a fairly uncommon car so you looked at a few online. It's only after this stimulus that you recognize there are more Volvos on the road than you thought (In fact, I bet the next time you are out driving on the road you will notice a Volvo. You've been primed by reading this example!).

Just like you need priming to recognize something you haven't noticed before, GPT-3 needs priming to generate new content that hasn't been seen before. The priming is done by providing a prompt as an input to the model. This could be the beginning part of an essay, a question, or even a partial block of code. All GPT-3 will do is determine what the best next word(s) is to complete your prompt and repeat that process until you tell it to stop.